

Apellidos:

Nombre:

Concurrencia y Paralelismo

Bloque I: Concurrencia

Junio 2023

1. Barrera [2.5 puntos]

Una barrera es una estructura de sincronización que para a un grupo de threads hasta que un cierto número ha llegado hasta ella. Los distintos threads llaman a una función `barrier()` que marca el punto de espera:

```
// N Threads corriendo en este código
...
barrier(b); // Todos los threads paran aquí hasta que n lleguen a este punto
...
```

Defina la estructura `barrier_t` y la función `barrier` de forma que paren a los threads hasta que `n` hayan llegado a la barrera.

```
typedef struct {
    int n; // Número de procesos que tienen que llegar a la barrera para que continuen.
    ...
} barrier_t;

void barrier(barrier_t *b) {
    ...
}
```

```
int mtx_lock(mtx_t *mtx);
int mtx_trylock(mtx_t *mtx);
int mtx_unlock(mtx_t *mtx);
int cnd_wait(cnd_t *cond, mtx_t *mtx);
int cnd_signal(cnd_t *cond);
int cnd_broadcast(cnd_t *cond);
```

Solución:

```
typedef struct {
    int n; // Número de procesos que tienen que llegar a la barrera para que continuen.
    int thr_reached; // starts at 0
    mtx_t lock;
    cnd_t barrier_wait;
} barrier_t;

void barrier(barrier_t *b) {
    mtx_lock(&b->lock);
    b->thr_reached++;
    if(b->thr_reached < b->n)
        cnd_wait(&b->barrier_wait, &b->lock);
    else
        cnd_broadcast(&b->barrier_wait);
    mtx_unlock(&b->lock);
}
```

2. Paso de Peatones [2.5 puntos]

Vamos a simular un paso peatonal con un semáforo que regula el paso tanto de vehículos como de peatones. El semáforo, cada peatón y cada coche está representado por un thread. La variable green indica quien tiene el semáforo en verde.

Los peatones pueden cruzar todos a la vez, pero los vehículos tienen que ir de uno en uno respetando su orden de llegada. Para marcar ese orden hay dos contadores compartidos, uno que indica cual es el número del último coche (last_car), y otro que indica qué coche es el siguiente en cruzar (first_car).

Como el semáforo cambia inmediatamente, tanto coches como peatones tienen que asegurarse de que el cruce está libre. Es decir, aunque tengan el semáforo en verde, los coches deberían esperar a que los peatones terminen de cruzar (y viceversa).

```
#define CARS 0
#define PEDESTRIANS 1
#define SEMAPHORE_TIME ...

struct crossing {
    int first_car; // Number of the first car that is waiting
    int last_car;  // Number of the last car that is waiting
    int green;     // Who has the green light right now
    mtx_t cross_m;
    cnd_t cars_wait, peds_wait;
};

int semaphore(void *ptr) {
    struct crossing *cross = ptr;

    while(true) {
        mtx_lock(&cross->cross_m);
        cross->green = (cross->green + 1) % 2;
        if(cross->green == CARS) cnd_broadcast(&cross->cars_wait);
        else cnd_broadcast(&cross->peds_wait);
        mtx_unlock(&cross->cross_m);
        sleep(SEMAPHORE_TIME);
    }
}

int pedestrian(void *ptr) {
    struct crossing *cross = ptr;
    // pedestrian enter

    do_cross(); // simulates the actual crossing.

    // Pedestrian exit
}

int car(void *ptr) {
    struct crossing *cross = ptr;
    // car enters

    do_cross(); // simulates the actual crossing.

    // car exits
}
```

Implemente las funciones pedestrian y car. Puede modificar la estructura crossing si lo necesita.

Solución

```
#define CARS 0
#define PEDESTRIANS 1
#define SEMAPHORE_TIME ...

struct crossing {
    int first_car; // Number of the first car that is waiting (starts at 0)
    int last_car; // Number of the last car that is waiting (starts at 0)
    int green; // Who has the green light right now
    mtx_t cross_m;
    int pedestrians; // starts at 0
    int cars; // starts at 0
    cond_t cars_wait, peds_wait;
};

int semaphore(void *ptr) {
    struct crossing *cross = ptr;

    while(true) {
        mtx_lock(&cross->cross_m);
        cross->green = (cross->green + 1) % 2;
        if(cross->green == CARS) cnd_broadcast(&cross->cars_wait);
        else cnd_broadcast(&cross->peds_wait);
        mtx_unlock(&cross->cross_m);
        sleep(SEMAPHORE_TIME);
    }
}

int pedestrian(void *ptr) {
    struct crossing *cross = ptr;

    mtx_lock(cross->cross_m);
    while(cross->green != PEDESTRIANS || cross->cars > 0)
        cnd_wait(&cross->peds_wait, &cross->cross_m);
    cross->pedestrians++;
    mtx_unlock(cross->cross_m);

    do_cross(); // simulates the actual crossing.

    mtx_lock(cross->cross_m);
    cross->pedestrians--;
    if(cross->pedestrians == 0 && cross->green == CARS)
        cnd_broadcast(cross->cars_wait);
    mtx_unlock(cross->cross);

    return 0;
}

int car(void *ptr) {
    struct crossing *cross = ptr;
    int myturn;

    mtx_lock(cross->cross_m);
    myturn = cross->last_car;
    cross->last_car++;
    while(cross->green != CARS || cross->cross->first_car != myturn || cross->pedestrians > 0)
        cnd_wait(&cross->cars_wait, &cross->cross_m);
    cross->cars = 1;
    mtx_unlock(cross->cross_m);

    do_cross(); // simulates the actual crossing.

    mtx_lock(cross->cross_m);
    cross->cars = 0;
    cross->first_car++;
    if(cross->green == CARS) cnd_broadcast(&cross->cars_wait);
    else cnd_broadcast(&cross->peds_wait);
    mtx_unlock(cross->cross_m);

    return 0;
}
```

EXAMEN DE CONCURRENCIA Y PARALELISMO

Bloque II: Paralelismo

APELLIDOS: _____ NOMBRE: _____

Convocatoria ordinaria

29 de mayo de 2023

AVISO: *No mezcléis en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregarán por separado.*

- [2.5p] 1. **Conceptos de paralelismo.** Un supermercado recibe diariamente los productos que deben ser organizados en los lineales para su venta. La central siempre envía un tráiler con **32 palets** del mismo volumen. El Supervisor de Recepción de Productos (SRP) es el encargado de que este proceso se realice eficientemente, y para ello gestiona un Equipo de Reposición de Lineales (ERL) de **8 trabajadores** entre los que se encuentra él mismo.
- Al comenzar la jornada, el SRP dedica **15 minutos** a recibir la mercancía, contrastando su hoja de pedido con el albarán que le proporciona el conductor del tráiler.
 - A continuación se descarga la mercancía. Hasta que el tráiler esté vacío, los miembros del ERL se alternan para acceder individualmente al tráiler y tardan **2 minutos** en descargar y transportar **uno de los palets** a la zona de almacenamiento. Solo disponen de una carretilla.
 - Cada miembro del ERL se encarga del procesamiento de los palets que ha descargado: anota su contenido en el Informe de Reposición de Mercancía (IRM) y mueve los productos a los lineales del supermercado. El tiempo de procesamiento de un palet varía en gran medida en función de los productos que contenga. Por ejemplo, un palet que contenga pocos artículos de gran tamaño se procesará en 3 minutos, mientras que un palet que contenga un gran número de artículos pequeños y posicionados en lineales lejanos demorará 20 minutos.
 - Debido a estas diferencias, cuando un miembro del ERL termina de procesar sus palets, comprueba si a algún compañero le queda algún palet por organizar, y asume ese trabajo.
 - Finalmente, el SRP necesita **10 minutos** para verificar que los productos incluidos en el IRM se corresponden con su hoja de pedido.
- a) [0.5p] Determina qué tipo de descomposición y asignación de tareas se ha hecho en este procedimiento.
- b) [1.25p] Una vez terminada la fase de procesamiento de palets, los trabajadores han tardado de media 40 minutos. El trabajador más rápido ha tardado 28 minutos, mientras que el más lento ha tardado 50. Calcula las siguientes métricas:
- Tiempo secuencial y paralelo
 - Aceleración
 - Eficiencia paralela.
 - Coste
 - Sobrecarga

c) **[0.75p]** Cuando se abre un palet, observando su mercancía es posible estimar el tiempo que llevará su procesamiento. El SRP propone una nueva estrategia: si se estima que el tiempo de procesamiento de un palet es demasiado alto, se puede dividir en 2 palets de menor tamaño, que serán procesados de forma independiente siguiendo el mismo procedimiento que el resto de palets.

- ¿Qué tipo de descomposición y asignación de tareas se realiza sobre el problema al incorporar esta nueva estrategia?
- Explica razonadamente si esta nueva estrategia es más o menos eficiente que la estrategia anterior. La justificación debe centrarse en los conceptos de “granularidad” y “balanceo de carga”.

[2.5p] 2. Diseño de algoritmos paralelos. Un conjunto de empresas del sector ha conseguido acceso a la base de datos de la FIC donde se guarda el histórico de las notas de todos los alumnos del grado en Ingeniería Informática desde que se impartió por primera vez. Concretamente, tienen interés en una tabla T de M filas donde cada una representa a un alumno, y N columnas donde cada una indica la nota para una determinada asignatura.

Estas empresas acceden a la tabla para saber cómo de adecuados son todos los antiguos alumnos del grado para un determinado puesto. Sin embargo, no todos los perfiles están interesados en los mismos conocimientos. Para un cierto perfil se debe crear un vector de ponderación V de N elementos que indique con un valor entre 0 y 1 el valor que se le da a cada asignatura. De esta forma, las empresas calculan una puntuación para cada alumno i de forma: $P_i = \sum_j (T_{i,j} \cdot V_j)$.

Una determinada empresa quiere desarrollar un código paralelo que acelere el cálculo de las puntuaciones y, así, tener ventaja competitiva sobre las demás por conocer antes al candidato idóneo. Hay que tener en cuenta que, para no levantar sospechas en la FIC con múltiples conexiones, solo un proceso (supongamos $P0$) puede tener acceso a la tabla T

(función *leeTabla(float * T)*) y, por simplicidad, ese será también el único que inicializará los valores del vector V (función *inicializaVector(float * V)*) y el que volcará todas las puntuaciones en un fichero o base de datos interna (función *vuelcaPuntuaciones(float * P)*). Cabe remarcar que en esta parte del código el objetivo es obtener y volcar las puntuaciones de todos los alumnos, no quedarse simplemente con el de puntuación más alta.

- a) **[0.5p]** Indica razonadamente qué tipo de descomposición y asignación de tareas usarías para desarrollar ese código paralelo.
- b) **[1.25p]** Escribe el código MPI asociado a esa solución usando funciones colectivas siempre que sea posible. Asume que la empresa pondrá a trabajar un número de procesos que será divisor de M y de N .
- c) **[0.75p]** Se desea modificar la aplicación de forma que, una vez obtenidas todas las puntuaciones, $P0$ buscará el alumno (número de fila en T) con mayor puntuación para contactar con él. En caso de empate a puntuaciones no importa cuál elija. Indica, de palabra, cómo modificarías el código del apartado anterior para que esta búsqueda del máximo también se realice en paralelo. Recuerda que el $P0$ necesitará igualmente volcar todo el vector de puntuaciones por si ese candidato rechaza el puesto y hay que contactar con el siguiente.

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm, MPI_Status *status)
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
             int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
              void *recvbuf, int recvcnt, MPI_Datatype recvtype,
              int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
              void *recvbuf, int recvcnt, MPI_Datatype recvtype,
              int root, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
              MPI_Datatype datatype, MPI_Op op,
              int root, MPI_Comm comm)
```

SOLUCIÓN

1. a) Es una descomposición de dominio, siendo el dominio el conjunto de palets. La asignación de tareas durante el procesamiento es dinámica descentralizada, porque la carga de trabajo se distribuye en tiempo de ejecución. Es posible mencionar **adicionalmente** una descomposición funcional sobre el problema completo y una asignación estática cíclica inicial.
b)
 - $T_{seq} = 15 + 2 \times 32 + 40 \times 8 + 10 = 409 \text{ min}$
 - $T_{par} = 15 + 2 \times 32 + 50 + 10 = 139 \text{ min}$
 - $A = 409/139 = 2,94$
 - $Ef = 2,94/8 \simeq 0,37 = 37 \%$
 - $Coste = 139 \times 8 = 1112 \text{ min}$
 - $Sobrecarga = 1112 - 409 = 703 \text{ min}$c)
 - Se trata una descomposición recursiva, pues las tareas se subdividen en tiempo de ejecución hasta que su carga de trabajo esté por debajo de un nivel crítico. La asignación sigue siendo dinámica descentralizada pues se sigue el mismo procedimiento que en el enunciado.
 - Es más eficiente porque se reduce la granularidad de la descomposición (grano más fino), generando tareas con un coste más homogéneo que, junto con la asignación dinámica propuesta, permite balancear mejor la carga de trabajo.
2. a) La descomposición sería de dominio, donde una tarea se asociaría al cálculo de cada valor P . La asignación sería estática porque se conocen de antemano el número de tareas y todas tienen el mismo coste computacional. Vale tanto una asignación bloque como cíclica porque para el cálculo de P_i solo se necesita conocer la fila i y la carga computacional es homogénea (ninguna opción tiene ventaja sobre la otra).

```
b) float *T, *V, *myT, *myP, *P;

V = (float *) malloc(sizeof(float) * N);
myT = (float *) malloc(sizeof(float) * M/numP * N);
myP = (float *) malloc(sizeof(float) * M/numP);
if(!rank) {
    T = (float *) malloc(sizeof(float) * M * N);
    P = (float *) malloc(sizeof(float) * M);
    leeTabla(T);
    inicializaVector(V);
}

MPI_Scatter(T, M/numP * N, MPI_FLOAT, myT, M/numP * N,
    MPI_FLOAT, 0, MPI_COMM_WORLD);
MPI_Bcast(V, N, MPI_FLOAT, 0, MPI_COMM_WORLD);

for(i = 0; i < M/numP; i++) {
    myP[i] = 0;
    for(j = 0; j < N; j++)
        myP[i] += myT[i * N + j] * V[j];
}

MPI_Gather(myP, M/numP, MPI_FLOAT, P, M/numP, MPI_FLOAT, 0, MPI_COMM_WORLD);

if(!rank)
    vuelcaPuntuaciones(P);
```

c) Se incluirá código para lo siguiente:

- En el bucle cada proceso va guardando la máxima puntuación de entre los alumnos que tiene asignados. También debe guardar cuál es el número de fila (id) del alumno que tiene ese máximo.
- Se hace un gather con los máximos valores teniendo $P0$ como raíz. $P0$ recorre ese array de máximos para saber cuál es la mayor puntuación. Si la mayor puntuación es la de la posición k significa que el proceso k es el que ha obtenido el máximo valor.
- Se hace otro gather con los ids de los máximos de cada proceso. El $P0$ sabe (por el punto anterior) que el proceso que obtuvo el máximo es k , así que para saber el id del alumno con la mejor puntuación solo tiene que coger el valor de la posición k en el vector resultado de este último gather.

IMPORTANTE: Una reducción con MPI_MAX en lugar del primer gather no vale porque no nos proporciona la información de qué proceso obtuvo el máximo, y no se podría obtener el consecuente id en el último paso. Sí sería válida con MPI_MAXLOC .